mra.py 整体模型科研式说明手册

2026-04-16

摘要

本文面向仓库当前版本的 `mra.py`，给出一份覆盖**完整流水线**的整体性 LaTeX 说明文档。报告不再分别拆解图专家、频域专家和采样率感知 Transformer，而是把它们放回同一条训练与推理闭环里分析：从原始多采样率 CSV、结构性缺失掩码、标准化和窗口构造开始，到双专家门控插补、采样率感知异常检测、多源分数融合、EWAF 平滑、阈值选择和最短异常时长约束结束。文中重点回答八类问题：第一，主程序真实执行链如何串联；第二，默认数据和默认参数下的张量形状是什么；第三，三种掩码如何在训练中协同工作；第四，双专家门控插补如何生成 $\boldsymbol{X}_{\text{complete}}$；第五，检测器如何把变量编码、采样率编码、相位编码和观测掩码合成为时间步 token；第六，五项损失分别更新哪些参数；第七，训练得到的模型权重和统计量如何继续作用于测试集；第八，最终输出文件各自对应哪一段计算结果。文中所有结论都严格对应当前代码实现，并补充若干实现级注意事项，例如 `gate_entropy` 只作分析不用作最终打分、`detector_loss` 虽然对目标使用了 `detach()` 但仍会反向更新插补器，以及默认指标函数中的 `fdr` 实际与 `recall` 使用相同公式这一代码事实。

关键词： 多采样率，缺失值插补，图学习，频域重构，Transformer 异常检测，分数融合

目录

1 文档范围与核心结论 **3**
- 1.1 分析范围 . . . 3
- 1.2 先给结论 . . . 3

2 任务定义与总体调用链 **4**
- 2.1 任务定义 . . . 4
- 2.2 主程序中的关键组成 . . . 4
- 2.3 训练与推理的真实调用链 . . . 5

3 数据、掩码、窗口与多采样率元数据 **5**
- 3.1 默认数据规模 . . . 5
- 3.2 标准化器的实际语义 . . . 5
- 3.3 窗口构造 . . . 6
- 3.4 三种掩码的关系 . . . 6
- 3.5 多采样率元数据推断 . . . 7

4 双专家门控插补主线 **7**
- 4.1 从输入窗口到完成窗口 . . . 7
- 4.2 PreImpute 的作用 . . . 8

4.3 图专家 8
4.4 频域专家 8
4.5 门控融合 9

5 采样率感知 Transformer 异常检测 9
5.1 检测器接收什么输入 9
5.2 变量、采样率与相位编码 9
5.3 时间步级 token 的构造 10
5.4 检测器异常分数 10

6 分数融合、平滑与异常判定 10
6.1 三类基础分数 10
6.2 shift_score 的来源 10
6.3 最终原始分数融合 11
6.4 EWAF 平滑与分段处理 11
6.5 阈值与最短异常时长约束 11

7 联合训练目标与参数更新 12
7.1 五项损失 12
7.2 各项损失更新谁 12
7.3 优化器与训练设置 12
7.4 参数规模 13

8 训练产物如何延续到测试阶段 13
8.1 测试阶段继续依赖哪些训练产物 13
8.2 完整序列重建 13
8.3 测试分数与预测保存 14
8.4 输出文件总表 14

9 实现细节与注意事项 14
9.1 图先验的真实来源 14
9.2 node_coords 在默认主线中基本不参与前向 15
9.3 门控熵不进入最终分数 15
9.4 指标函数里的 `fdr` 需要谨慎解读 15
9.5 测试标签是固定分段标签 15

10 总结 15

1 文档范围与核心结论

1.1 分析范围

本文覆盖 `mra.py` 主程序中与完整方法闭环直接相关的全部主线：

- 数据读取、结构性缺失掩码构造、观测值标准化、窗口化与测试标签切分；
- 多采样率元数据推断，即 `rate_id` 与 `stride` 的生成；
- `PreImpute`、图专家、频域专家、门控融合和完整窗口 $\boldsymbol{X}_{\text{complete}}$ 的生成；
- `SamplingAwareTransformerAD` 的时间步级编码与重构式异常检测；
- `detector_score`、`disagreement_score`、`shift_score` 的构造与最终分数融合；
- EWAF 平滑、训练阈值选择、最短异常持续长度约束和最终分类指标；
- 模型权重、标准化器、训练分数统计量和输出文件在测试阶段的延续作用。

仓库中已经存在三份专题文档：图学习说明、频域模块说明和采样率感知检测器说明。本文可以视为它们的**总览版与总装版**，强调模块之间如何在同一份代码里协同工作。

1.2 先给结论

结合代码实现与默认数据配置，可以先给出十条最关键的结论：

- `mra.py` 实现的是一条完整的“**插补 + 异常检测**”联合流水线，而不是只做某一段模块实验。
- 默认数据是 4100×18 的多采样率时序，18 个变量会被自动识别为三组采样步长：1、3、5。
- 训练时同时存在三种掩码：结构性缺失掩码 $\boldsymbol{M}_s$、随机 holdout 掩码 $\boldsymbol{M}_h$ 和模型输入掩码 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$。
- 插补器不是单专家，而是**图 GCN 专家 + 频域专家 + 采样率感知门控**的双专家融合结构。
- 图专家和频域专家都不直接看原始零填充窗口，而是先看 `PreImpute.fill` 生成的平滑种子 $\boldsymbol{X}_{\text{seed}}$。
- 检测器接收的不是原始缺失窗口，而是门控补全后的 $\boldsymbol{X}_{\text{complete}}$，因此它学习的是“正常完成窗口”的重构性。
- 最终异常分数由三部分组成：窗口重构误差、双专家分歧和完成窗口的分布偏移；`gate_entropy` 只被记录，不参与最终判定。
- 五项损失共享一个 AdamW 优化器做联合训练，`detector_loss` 虽然使用了 `detach()` 目标，仍会通过 $\boldsymbol{X}_{\text{complete}}$ 反向更新插补器。
- 测试阶段不仅使用训练好的神经网络参数，还继续依赖训练集估计出的标准化均值方差、距离先验、`rate_id`、`stride`、分数归一化参数和阈值。
- 默认配置下模型总参数量约为 693,260，其中检测器参数约 656,466，插补器参数约 36,794，整体计算重心明显在检测器侧。

2 任务定义与总体调用链

2.1 任务定义

给定带有原生缺失的多采样率时序

$$\boldsymbol{X}^{\mathrm{raw}} \in \mathbb{R}^{T_{\mathrm{all}} \times N}, \tag{1}$$

以及对应的结构性缺失掩码

$$\boldsymbol{M}_s \in \{0,1\}^{T_{\mathrm{all}} \times N}, \tag{2}$$

代码需要完成两件事：

1. 在窗口级完成多采样率缺失值插补，得到完整时序估计；
2. 在测试窗口上给出异常分数与二分类异常判定。

因此 `mra.py` 的目标不是简单的插补误差最小化，而是

> 先学习一个对多采样率缺失模式鲁棒的完成窗口，再用这个完成窗口驱动无监督异常检测。

2.2 主程序中的关键组成

组成部分	作用
`parse_args`	定义数据路径、窗口长度、训练轮数、门控和检测器超参数、阈值和平滑参数等命令行入口。
`infer_rate_metadata`	根据训练集缺失率推断每个变量的采样步长 `stride` 与采样率类别 `rate_id`。
`PreImpute`	对窗口做双向填充，生成平滑时域种子 $\boldsymbol{X}_{\mathrm{seed}}$。
`TwoExpertGatedImputer`	并行调用图专家和频域专家，再结合采样率嵌入做逐位置门控融合。
`SamplingAwareTransformerAD`	对完成窗口做采样率感知重构，输出检测器重构结果。
`FusionAnomalyModel`	把插补器与检测器封装成完整前向图。
`compute_losses`	定义五项损失及其加权总和。
`train_model`	负责随机 holdout、前向、反向传播、梯度裁剪与 AdamW 更新。
`reconstruct_full_sequence`	用训练后模型恢复训练集和测试集的完整序列，并保存门控权重与邻接矩阵样本。
`collect_window_statistics`	统计 `detector_score`、`disagreement_score`、`gate_entropy`。
`compute_distribution_shift_score`	基于完成窗口的统计特征构造 `shift_score`。
`combine_scores`	把多源分数标准化后组合为最终原始异常分数。
`apply_ewaf_by_segments`	对训练和测试分数按段执行指数加权平滑。
`choose_threshold`	基于训练分数选择阈值。
`apply_min_anomaly_duration`	去除持续长度不足的异常片段。

2.3 训练与推理的真实调用链

主程序的训练主线可以写成

$$\begin{aligned}\texttt{main} &\to \texttt{load_csv_glob_with_mask} \to \texttt{ObservedStandardScaler}\\ &\to \texttt{build_windows} \to \texttt{infer_rate_metadata}\\ &\to \texttt{FusionAnomalyModel} \to \texttt{train_model}\\ &\to \texttt{compute_losses} \to \texttt{loss.backward} \to \texttt{AdamW.step}.\end{aligned} \tag{3}$$

推理与后处理主线可以写成

$$\begin{aligned}\texttt{reconstruct_full_sequence} &\to \texttt{collect_window_statistics} \to \texttt{compute_distribution_shift_scores}\\ &\to \texttt{combine_scores} \to \texttt{apply_ewaf_by_segments}\\ &\to \texttt{choose_threshold} \to \texttt{apply_min_anomaly_duration} \to \texttt{metrics}.\end{aligned} \tag{4}$$

这说明 `mra.py` 中并不存在“先独立训练插补器，再冻结检测器”的两阶段过程；当前实现是**单次联合训练 + 训练后统一推理**。

3 数据、掩码、窗口与多采样率元数据

3.1 默认数据规模

仓库默认输入文件为：

- 训练集：`data/train/train_1.csv`；
- 测试集：`data/test/test_C5_1.csv`。

两者都具有相同形状：

$$\boldsymbol{X}_{\text{raw}}^{\text{train}} \in \mathbb{R}^{4100\times 18}, \tag{5}$$

$$\boldsymbol{X}_{\text{raw}}^{\text{test}} \in \mathbb{R}^{4100\times 18}. \tag{6}$$

训练集和测试集的总体缺失率均约为

$$0.4889. \tag{7}$$

从 CSV 内容可以直接看出三组采样频率：

- 第 1 到 6 列几乎每个时间步都有观测；
- 第 7 到 12 列大约每 3 个时间步观测一次；
- 第 13 到 18 列大约每 5 个时间步观测一次。

3.2 标准化器的实际语义

`ObservedStandardScaler` 只用训练集的**观测值**拟合均值和标准差。对每个变量 n，记均值和标准差分别为 μ_n 与 σ_n，则标准化结果为

$$X_{t,n} = \begin{cases} \dfrac{X_{t,n}^{\text{raw}} - \mu_n}{\sigma_n}, & M_{s,t,n} = 0,\\ 0, & M_{s,t,n} = 1. \end{cases} \tag{8}$$

因此标准化后的数值零包含两种可能语义：

- 原始观测值恰好标准化为零；
- 原始位置缺失，用观测均值填充后标准化为零。

这就是为什么后续模型必须始终联合使用数据张量和掩码张量。

3.3 窗口构造

代码中同时构造三类窗口：

- `build_windows`：用于训练插补和全序列重建，支持前端补齐；
- `build_standard_windows`：用于训练分数统计，不做前端补齐；
- `build_prompt_test_windows`：用于测试分数统计，并生成固定的正常/异常标签。

默认参数 `seq_len` $= 50$、`stride` $= 1$ 时，窗口形状为

$$\boldsymbol{W}_{\text{imp}}^{\text{train}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{9}$$

$$\boldsymbol{W}_{\text{imp}}^{\text{test}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{10}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{train}} \in \mathbb{R}^{4001\times 50\times 18}, \tag{11}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{test}} \in \mathbb{R}^{4000\times 50\times 18}. \tag{12}$$

测试标签由 `build_prompt_test_windows` 直接生成，默认切分为：

$$\underbrace{0,\ldots,0}_{2000\text{ 个窗口}},\ \underbrace{1,\ldots,1}_{2000\text{ 个窗口}}. \tag{13}$$

3.4 三种掩码的关系

对一个训练 batch，代码中出现三种掩码：

- $\boldsymbol{M}_s$：结构性缺失掩码，来自原始 CSV 的 `NaN`；
- $\boldsymbol{M}_h$：训练时从观测点中随机采样出的 holdout 掩码；
- $\boldsymbol{M}$：模型实际输入掩码，满足 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$。

于是模型输入窗口为

$$\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M}). \tag{14}$$

`sample_holdout_mask` 还包含一个关键保底逻辑：只要某个样本窗口中存在可观测点，若随机采样没有选中任何 holdout，代码会强制随机选中一个可观测位置作为训练目标。这保证了插补损失不会在该样本上失去监督。

3.5 多采样率元数据推断

`infer_rate_metadata` 先计算每个变量的缺失率和观测率

$$r_n = 1 - \frac{1}{T_{\text{all}}} \sum_{t=1}^{T_{\text{all}}} M_{s,t,n}, \tag{15}$$

再推断采样步长

$$s_n = \text{round}\left(\frac{1}{\max(r_n, 10^{-6})} \right). \tag{16}$$

默认数据会得到三组变量：

变量组	列索引	缺失率	观测率	推断步长 s_n	`rate_id`
高频组	1–6	0.0000	1.0000	1	0
中频组	7–12	0.6666	0.3334	3	1
低频组	13–18	0.8000	0.2000	5	2

因此主程序默认使用

$$\boldsymbol{stride} = [1,1,1,1,1,1,3,3,3,3,3,3,5,5,5,5,5,5], \tag{17}$$

$$\boldsymbol{rate_id} = [0,0,0,0,0,0,1,1,1,1,1,1,2,2,2,2,2,2]. \tag{18}$$

4 双专家门控插补主线

4.1 从输入窗口到完成窗口

双专家插补器的核心计算可以概括为

$$\begin{aligned}
\boldsymbol{X}_{\text{seed}} &= \text{PreImpute}(\boldsymbol{X}_{\text{input}}, \boldsymbol{M}), \\
\boldsymbol{X}_{\text{gcn}}, \boldsymbol{A} &= f_{\text{graph}}(\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}), \\
\boldsymbol{X}_{\text{freq}} &= f_{\text{freq}}(\boldsymbol{X}_{\text{seed}}), \\
\boldsymbol{\Pi} &= g(\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}, \boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{rate_id}), \\
\boldsymbol{X}_{\text{imputed}} &= \Pi_{:,:,:,0} \odot \boldsymbol{X}_{\text{gcn}} + \Pi_{:,:,:,1} \odot \boldsymbol{X}_{\text{freq}}, \\
\boldsymbol{X}_{\text{complete}} &= \boldsymbol{M} \odot \boldsymbol{X}_{\text{imputed}} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{\text{input}}.
\end{aligned} \tag{19}$$

其中：

- $\boldsymbol{X}_{\text{seed}}$ 是预插补种子；
- $\boldsymbol{X}_{\text{gcn}}$ 是图专家输出；
- $\boldsymbol{X}_{\text{freq}}$ 是频域专家输出；
- $\boldsymbol{\Pi} \in \mathbb{R}^{B \times T \times N \times 2}$ 是逐时间步、逐变量的专家权重。

4.2 PreImpute 的作用

`PreImpute.fill` 会对每个变量分别做一次正向填充和一次反向填充：

- 若缺失位置前后都有可用值，则取双向均值；
- 若只有一侧有值，则取单侧填充值；
- 若两侧都没有值，则保持为零。

这一步有两个直接作用：

1. 给图专家提供比零填充更稳定的窗口统计量；
2. 给频域专家提供更平滑的时域信号，减少直接 FFT 到零块带来的频谱伪影。

4.3 图专家

图专家由 `GraphGCNReconstructor` 实现，内部又包含动态图学习器 `GraphLearner`。它会从当前窗口提取每个变量的动态上下文：

$$\text{dynamic}_n = [\text{mean}_n, \text{std}_n, \text{last}_n, \text{missing_ratio}_n]\,. \tag{20}$$

再与可学习的静态上下文拼接，经过 MLP 得到节点表示，进而构造节点对特征并输出边 logits。最终邻接打分由三部分叠加：

$$\text{logits}_{ij} = \text{edge_mlp}_{ij} + \lambda_p \cdot \text{prior}_{ij} + \lambda_s \cdot \text{cosine}_{ij}. \tag{21}$$

其中 prior_{ij} 默认不是来自外部物理坐标，而是由**训练数据轨迹 + 观测模式**自动构造的欧氏距离先验。之后还会执行对角增强、softmax、对称化和行归一化，得到

$$\boldsymbol{A} \in \mathbb{R}^{B\times N\times N}. \tag{22}$$

需要特别指出：邻接矩阵是**按窗口动态生成**的，但在同一窗口内部会对所有时间步共享，因此图卷积主要建模的是“同一时刻不同变量之间的关系”，时间信息主要通过窗口统计量进入构图。

4.4 频域专家

频域专家由 `FrequencyOnlyReconstructor` 实现，先沿时间维执行实数 FFT：

$$\boldsymbol{Z} = \mathcal{F}_r(\boldsymbol{X}_{\text{seed}}), \tag{23}$$

再把实部与虚部拼接成频谱描述向量，经过注意力支路和频谱增强支路后构造复数残差，最后通过逆 FFT 回到时域。该专家还接了一个跨变量的 `output_head` 残差修正。

和图专家不同，频域专家不显式读取掩码或采样率嵌入；它对多采样率缺失结构的感知主要来自 $\boldsymbol{X}_{\text{seed}}$ 本身。

4.5 门控融合

门控网络对每个时间步、每个变量构造如下特征：

$$[x_{\text{seed}}, m, x_{\text{gcn}}, x_{\text{freq}}, x_{\text{gcn}} - x_{\text{seed}}, x_{\text{freq}} - x_{\text{seed}}, e_{\text{rate}}]. \tag{24}$$

其中前六项是标量特征，$e_{\text{rate}} \in \mathbb{R}^8$ 是采样率嵌入，因此默认门控输入维度为

$$6 + 8 = 14. \tag{25}$$

门控 MLP 输出两个 logits，经 softmax 后得到

$$\Pi_{t,n,0} + \Pi_{t,n,1} = 1. \tag{26}$$

这说明当前实现中的专家融合是**位置相关、变量相关且显式感知采样率类别**的。

5 采样率感知 Transformer 异常检测

5.1 检测器接收什么输入

`SamplingAwareTransformerAD` 的输入不是原始缺失窗口，而是

$$\boldsymbol{X}_{\text{complete}} \in \mathbb{R}^{B \times T \times N}, \tag{27}$$

以及结构性观测掩码

$$\boldsymbol{O} = 1 - \boldsymbol{M}_s. \tag{28}$$

这意味着检测器学习的是“正常完成窗口”的可重构性；原生缺失点本身不会直接进入检测器损失。

5.2 变量、采样率与相位编码

检测器会为每个变量拼接五类信息：

1. 当前完成值 $x_{t,n}$；
2. 当前观测指示 $o_{t,n}$；
3. 变量编号嵌入；
4. 采样率类别嵌入；
5. 由步长推导的相位嵌入。

相位定义为

$$\phi_{t,n} = \frac{\text{mod}(t, s_n)}{s_n}, \qquad \phi_{t,n} \in [0, 1). \tag{29}$$

默认配置下：

- 变量嵌入维度为 8；
- 采样率嵌入维度为 8；
- 相位 MLP 输出维度为 8；
- 单变量拼接输入维度为 $1 + 1 + 8 + 8 + 8 = 26$；
- 单变量投影后维度 `per_variable_dim` $= 16$。

5.3 时间步级 token 的构造

检测器不是为每个变量单独建立 token，而是把同一时间步的全部变量特征拼成一个时间 token。默认输入维度为

$$\texttt{token_input_dim} = N + N + N \times \texttt{per_variable_dim} = 18 + 18 + 18 \times 16 = 324. \tag{30}$$

经过线性层投影后得到

$$\boldsymbol{T} \in \mathbb{R}^{B \times 50 \times 128}, \tag{31}$$

再叠加正弦位置编码，送入三层 Transformer Encoder。最终重构头输出一个修正量 Δ，并以残差形式得到检测器重构结果：

$$\widehat{\boldsymbol{X}}_{\text{det}} = \boldsymbol{X}_{\text{complete}} + \Delta. \tag{32}$$

5.4 检测器异常分数

在窗口级统计中，检测器分数定义为结构性观测位置上的平均平方误差：

$$s_{\text{det}} = \frac{\left\| (\widehat{\boldsymbol{X}}_{\text{det}} - \boldsymbol{X}_{\text{complete}}) \odot \boldsymbol{O} \right\|_F^2}{\sum \boldsymbol{O}}. \tag{33}$$

因此它反映的是“完成窗口在观测位置上的可重构程度”，而不是对原始缺失点的直接重构误差。

6 分数融合、平滑与异常判定

6.1 三类基础分数

`collect_window_statistics` 和 `compute_distribution_shift_scores` 会构造三条真正进入最终判定主线的分数：

1. 检测器分数 s_{det}；
2. 专家分歧分数
$$s_{\text{gap}} = \text{mean}|\boldsymbol{X}_{\text{gcn}} - \boldsymbol{X}_{\text{freq}}|; \tag{34}$$
3. 分布偏移分数 s_{shift}。

此外代码还会额外记录

$$s_{\text{entropy}} = -\,\text{mean} \sum_k \Pi_k \log \Pi_k, \tag{35}$$

即 `gate_entropy`。但它只被写入 `test_predictions.csv`，**不参与最终异常打分**。

6.2 shift_score 的来源

`shift_score` 不是神经网络输出，而是对完成窗口做手工统计后得到的分布偏移分数。代码会从四个变量块

$$[1{:}6],\ [7{:}12],\ [13{:}18],\ [1{:}18] \tag{36}$$

提取窗口均值和窗口标准差，再加上

- 一阶差分绝对值均值；
- 低频 FFT 幅值均值。

默认形成一个 108 维的窗口特征向量，再用训练集均值和标准差做 z-score 标准化，最后取均方根得到 s_{shift}。

6.3 最终原始分数融合

三种分数先分别用训练集统计量标准化，再按如下方式组合：

$$s_{\text{raw}} = |z_{\text{det}}| + \lambda_{\text{gap}} z_{\text{gap}} + \lambda_{\text{shift}} z_{\text{shift}}. \tag{37}$$

默认权重为

$$\lambda_{\text{gap}} = 0.25, \qquad \lambda_{\text{shift}} = 1.0. \tag{38}$$

这里对检测器分数取绝对值，意味着代码默认同时关注“高于均值”和“低于均值”的重构偏离。

6.4 EWAF 平滑与分段处理

训练分数直接整体做 EWAF：

$$\widetilde{s}_1 = s_1, \qquad \widetilde{s}_t = \alpha s_t + (1-\alpha)\widetilde{s}_{t-1}. \tag{39}$$

默认 $\alpha = 0.3$。测试分数则会先按标签连续段切成两段，长度默认为

$$[2000, 2000], \tag{40}$$

再对每一段分别执行 EWAF，避免跨段平滑引入泄漏。

6.5 阈值与最短异常时长约束

阈值通过 `choose_threshold` 的 `gaussian_quantile_max` 模式得到：

$$\tau = \max\left(\mu_{\text{train}} + k\sigma_{\text{train}},\ Q_q(\widetilde{s}^{\text{train}})\right), \tag{41}$$

默认

$$k = 2.0, \qquad q = 0.95. \tag{42}$$

先得到阈值化预测

$$\widehat{y}_t^{\text{thr}} = \mathbb{I}(\widetilde{s}_t^{\text{test}} \geq \tau), \tag{43}$$

再执行最短异常持续长度约束。默认 `min_anomaly_duration` $= 50$，因此任何长度小于 50 的连续异常段都会被抑制为正常。

7 联合训练目标与参数更新

7.1 五项损失

对一个训练 batch，代码定义了五项损失：

$$\mathcal{L}_{\text{fusion}} = \text{MSE}_{\boldsymbol{M}_h}(\boldsymbol{X}_{\text{imputed}}, \boldsymbol{X}_{\text{true}}), \tag{44}$$

$$\mathcal{L}_{\text{gcn}} = \text{MSE}_{\boldsymbol{M}_h}(\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{true}}), \tag{45}$$

$$\mathcal{L}_{\text{freq}} = \text{MSE}_{\boldsymbol{M}_h}(\boldsymbol{X}_{\text{freq}}, \boldsymbol{X}_{\text{true}}), \tag{46}$$

$$\mathcal{L}_{\text{det}} = \text{MSE}_{\boldsymbol{O}}(\widehat{\boldsymbol{X}}_{\text{det}}, \text{sg}(\boldsymbol{X}_{\text{complete}})), \tag{47}$$

$$\mathcal{L}_{\text{graph}} = \text{mean}(\text{diag}(\boldsymbol{A}) - \gamma)^2 . \tag{48}$$

其中：

- $\text{MSE}_{\boldsymbol{M}_h}$ 表示只在 holdout 位置计算均方误差；
- $\text{MSE}_{\boldsymbol{O}}$ 表示只在结构性观测位置计算均方误差；
- $\text{sg}(\cdot)$ 表示 `detach()`；
- $\gamma = \texttt{diag_target} = 0.25$。

总损失为

$$\mathcal{L} = 1.0\,\mathcal{L}_{\text{fusion}} + 0.4\,\mathcal{L}_{\text{gcn}} + 0.4\,\mathcal{L}_{\text{freq}} + 0.5\,\mathcal{L}_{\text{det}} + 0.1\,\mathcal{L}_{\text{graph}}. \tag{49}$$

7.2 各项损失更新谁

损失项	主要更新对象
$\mathcal{L}_{\text{fusion}}$	图专家、频域专家、门控网络、采样率嵌入。
$\mathcal{L}_{\text{gcn}}$	图学习器与 GCN 重构层。
$\mathcal{L}_{\text{freq}}$	频域 FFT 增强支路与频域输出头。
$\mathcal{L}_{\text{det}}$	检测器参数；同时通过 $\boldsymbol{X}_{\text{complete}}$ 反向更新图专家、频域专家和门控。
$\mathcal{L}_{\text{graph}}$	图学习器中的邻接矩阵相关参数。

虽然 $\mathcal{L}_{\text{det}}$ 的目标张量被 `detach()`，但输入张量 $\boldsymbol{X}_{\text{complete}}$ 没有截断，因此这项损失仍会改变插补器。这是当前实现最容易被误解的地方之一。

7.3 优化器与训练设置

训练使用

- 优化器：AdamW；
- 学习率：10^{-3}；
- 权重衰减：10^{-4}；
- 梯度裁剪：$\|\nabla\|_2 \leq 1.0$；
- 默认轮数：12；

- 默认批大小：64。

这套配置由 `train_model` 统一施加到整个联合模型，而不是对不同模块分别使用不同优化器。

7.4 参数规模

在默认结构下，模型参数量约为：

$$\text{Total} = 693{,}260, \tag{50}$$

$$\text{Imputer} = 36{,}794, \tag{51}$$

$$\text{Detector} = 656{,}466. \tag{52}$$

进一步拆分插补器：

$$\text{Graph expert} = 7{,}206, \tag{53}$$

$$\text{Freq expert} = 24{,}314, \tag{54}$$

$$\text{Gate-related} = 5{,}274. \tag{55}$$

这说明整体模型的大部分参数预算实际上放在检测器上。

8 训练产物如何延续到测试阶段

8.1 测试阶段继续依赖哪些训练产物

测试阶段不是“只拿一个 `model.pt` 就够了”，而是至少继续依赖以下七类训练期产物：

1. 标准化器的均值与标准差；
2. 图学习先验矩阵；
3. `rate_id`；
4. `stride`；
5. 模型权重；
6. 各类训练分数的均值和标准差；
7. 训练分数导出的阈值。

这也是为什么主程序在保存模型时，会把 `rate_id`、`stride` 和 `loss_weights` 一起写入 `model.pt`。

8.2 完整序列重建

`reconstruct_full_sequence` 会对训练集和测试集的前端补齐窗口逐批前向，并取每个窗口**最后一个时间步**的完成值作为该时刻的重建结果。因此长度为 4100 的前端补齐窗口序列恰好可以恢复出长度为 4100 的完整时序。

随后代码会执行反标准化，分别生成：

- `train_completed.csv`；
- `test_completed.csv`。

8.3 测试分数与预测保存

测试阶段最终会形成一个包含多列信息的 test_predictions.csv，主要字段包括：

- label；
- detector_score；
- disagreement_score；
- gate_entropy；
- shift_score；
- raw_final_score；
- final_score；
- threshold_prediction；
- prediction。

因此该文件同时保留了“融合前”和“后处理后”的分数信息，便于后续分析。

8.4 输出文件总表

输出文件	含义
model.pt	训练后的模型权重，以及参数配置、rate_id、stride、损失权重。
train_completed.csv	训练集的完整序列重建结果。
test_completed.csv	测试集的完整序列重建结果。
train_gate_weights.csv	训练集上每个变量的平均图专家/频域专家门控权重。
test_gate_weights.csv	测试集上每个变量的平均图专家/频域专家门控权重。
train_adjacency_heatmap.png	训练阶段邻接矩阵平均热图。
train_adjacency_heatmap_offdiag.png	去除对角线后的邻接热图。
training_curve.png	总损失、插补损失和检测损失曲线。
anomaly_scores.png	测试分数、阈值和正常/异常分界可视化。
test_predictions.csv	测试窗口级预测与多源分数明细。
metrics.json	指标、阈值、平滑参数、分数统计量和采样率元数据汇总。

9 实现细节与注意事项

9.1 图先验的真实来源

若用户没有显式提供 --physical-coords-path 或 --physical-distance-path，主程序会调用

$$\texttt{resolve_distance_prior_matrix(train_scaled, train_mask)} \tag{56}$$

自动生成一个基于训练集轨迹与观测模式的欧氏距离先验。因此默认情况下，图先验是**数据驱动的训练集先验**，而非真实物理拓扑。

9.2 node_coords 在默认主线中基本不参与前向

GraphLearner 内部虽然定义了可学习的 node_coords，但只在没有传入 physical_distance_matrix 时才会用于构造先验。当前 mra.py 主程序总会先解析出一个距离先验矩阵并传入图专家，因此默认主线下 node_coords 实际上不参与邻接前向。

9.3 门控熵不进入最终分数

gate_entropy 常常容易被误读为最终分数的一部分。实际上它仅在 collect_window_statistics 中被记录，并写入预测明细 CSV，后续 combine_scores 并不会读取它。

9.4 指标函数里的 fdr 需要谨慎解读

compute_binary_classification_metrics 里

$$\texttt{fdr} = \frac{TP}{TP + FN}, \tag{57}$$

与代码中的 recall 使用了相同公式。因此若把该字段直接解释成传统统计意义上的 false discovery rate，会与实际代码不一致。写论文或对外汇报时应以代码实现为准，或单独修正命名。

9.5 测试标签是固定分段标签

默认测试标签并不是从测试 CSV 中读取的外部标注，而是由 build_prompt_test_windows 根据默认切分参数直接生成的固定分段标签。若后续数据组织方式发生变化，这一部分需要同步调整。

10 总结

mra.py 的核心价值不在于某一个单独模块，而在于它把多采样率缺失建模、图关系学习、频域重构、采样率感知检测、多源异常分数融合和后处理判定组织成了一条完整闭环。对当前代码来说，最值得把握的主线是：

> 结构性缺失与随机 holdout 共同定义训练任务，双专家门控插补先生成可靠的完成窗口，采样率感知 Transformer 再在完成窗口上学习正常行为，最后结合专家分歧和分布偏移形成更稳健的异常分数。

如果把这份总报告与仓库已有的三份专题文档结合使用，就可以同时回答两个层面的问题：一方面，系统整体是怎样从输入走到最终判定的；另一方面，每个关键子模块内部又具体做了什么。